# Compiler Construction

The 80/20 Revision Guide

Part IB — Cambridge CST

## How this guide is organised

The Compiler Construction paper (Paper 4) sets two questions per year. Across **eight years of past papers (2018–2025, 16 questions)** the structure is remarkably stable: **question 1 is always lexing / parsing / grammar work**; **question 2 is always back-end / runtime / optimisation work**. Topic frequencies:

| Topic | Appearances | Share |
|---|---|---|
| *Front-end (Q1 territory):* | | |
| Parsing techniques (LL, LR, SLR) | 5 | 31% |
| Grammar transformations | 3 | 19% |
| Lexical analysis | 3 | 19% |
| Syntax analysis | 2 | 13% |
| **Total Q1-ish (front-end)** | **8** | **100% of Q1s** |
| *Back-end (Q2 territory):* | | |
| Runtime systems and VMs | 4 | 25% |
| Code generation and optimisation | 3 | 19% |
| Intermediate code and translators | 3 | 19% |
| Error handling and exceptions | 3 | 19% |
| Tail call optimisation | 2 | 13% |
| Memory management / garbage collection | 2 | 13% |
| Type systems and strong typing | 2 | 13% |
| Data types and structures | 2 | 13% |

**80/20 verdict. Lexing + parsing + grammar transformations are guaranteed to appear and account for ~50% of the marks.** The other half is split across the back-end topics — but **runtime systems / VMs and intermediate code together hit ~50% of all Q2s**, with GC, exceptions, and TCO each appearing every few years.

This document goes in that order: **must-know first, then important, then context**.

---

# 1 The Big Three of the Front End (50% of marks)

## 1.1 Lexical analysis: RE → NFA → DFA

**Exam-critical**

The most-asked sub-topic. You must be able to: (a) draw an NFA for a given regex using Thompson's construction, (b) convert it to a DFA via the subset construction, (c) explain $\epsilon$-closure, (d) state why the longest-match rule matters, (e) sketch how DFA minimisation works.

### The job of a lexer

Lexing converts a sequence of *characters* into a sequence of *tokens*, each annotated with a token class and (optionally) an attribute (the identifier name, the integer value, etc.). The lexer is specified as a

list of (regex, token) pairs:

```
if                  -> IF
then                -> THEN
[a-zA-Z][a-zA-Z0-9]* as s -> IDENT s
[0-9]+               as n -> INT (int_of_string n)
[ \t\n]+             -> SKIP
```

### Regular expressions, formally

Regexes over alphabet $\Sigma$:

$$e \ ::= \ \emptyset \mid \epsilon \mid a \mid e \vee e \mid ee \mid e^*$$

The language $L(e)$ is defined inductively:

$$L(\emptyset) = \{\}, \quad L(\epsilon) = \{\epsilon\}, \quad L(a) = \{a\}$$

$$L(e_1 \vee e_2) = L(e_1) \cup L(e_2), \quad L(e_1 e_2) = \{w_1 w_2 \mid w_1 \in L(e_1), w_2 \in L(e_2)\}$$

$$L(e^*) = \bigcup_{n \geq 0} L(e^n)$$

### Thompson's construction: regex → NFA

A recursive procedure: each regex compiles to an NFA with one start and one accept state, parameterised by sub-NFAs.

| Regex | NFA fragment |
|---|---|
| $a$ | $\circ \xrightarrow{a} \bullet$ |
| $\epsilon$ | $\circ \xrightarrow{\epsilon} \bullet$ |
| $e_1 e_2$ | connect $e_1$'s accept to $e_2$'s start via $\epsilon$ |
| $e_1 \vee e_2$ | new start $\to_\epsilon$ both starts; both accepts $\to_\epsilon$ new accept |
| $e^*$ | new start $\to_\epsilon$ {old start, new accept}; old accept $\to_\epsilon$ {old start, new accept} |

Result: an NFA with at most $2 \cdot |e|$ states, exactly one accept state, where the only outgoing transitions of each state are either a single labelled edge or at most two $\epsilon$-edges.

### Subset construction: NFA → DFA

The DFA state corresponds to a *set* of NFA states (intuition: "the set of all NFA states we could be in, given the input so far").

**$\epsilon$-closure** of a set $S$: all NFA states reachable from any $q \in S$ via $\epsilon$-edges (including $S$ itself). This is exactly a transitive closure.

**Algorithm.**

- Start state: $\epsilon$-closure($\{q_0\}$).
- For each DFA state $S$ and input symbol $a$: the next state is $\epsilon$-closure($\{q' \mid q' \in \delta(q, a), q \in S\}$).
- Accept any DFA state containing an NFA accept state.

**Worst case**: exponential blow-up ($2^n$ DFA states for $n$ NFA states), but for practical lexer regexes the DFA stays modest.

### Longest match & priority

A real lexer has multiple regexes (one per token). Rules:

- **Longest match (maximal munch):** prefer the longest possible token, e.g. `ifx` is one `IDENT`, not `IF` followed by `IDENT x`.
- **Rule priority:** on ties, the regex listed first wins, e.g. `if` matches both the `IF` keyword regex and the `IDENT` regex; keywords go first.

### DFA minimisation (briefly)

Two states are **equivalent** iff they agree on accept-status and, for every input symbol, transition to equivalent states. The standard algorithm partitions states by "distinguishable behaviour" and yields a unique (up-to-renaming) minimal DFA.

## 1.2 LL(1) parsing

**Exam-critical**

The most testable parser flavour. You must be able to: compute FIRST and FOLLOW sets, build an LL(1) parsing table, run the parser on a given input, identify when a grammar is not LL(1) and apply standard transformations (left-recursion elimination, left factoring) to make it so.

### Recursive descent and predictive parsing

A recursive-descent parser has one mutually-recursive function per non-terminal. **Predictive parsing** is the deterministic case — the next $k$ tokens of lookahead uniquely determine which production to apply. LL($k$) = Left-to-right scan, Leftmost derivation, $k$ tokens of lookahead. We'll focus on $k = 1$.

### FIRST

FIRST($\alpha$) = the set of terminals that can appear at the start of any string derivable from $\alpha$, plus $\epsilon$ if $\alpha \Rightarrow^* \epsilon$.

**Compute FIRST iteratively until fixed point:**

- For terminal $a$: FIRST($a$) = $\{a\}$.
- For production $A \to X_1 X_2 \cdots X_n$: add FIRST($X_1$) $\setminus \{\epsilon\}$ to FIRST($A$). If $\epsilon \in$ FIRST($X_1$), also add FIRST($X_2$) $\setminus \{\epsilon\}$, and so on; if all $X_i$ can derive $\epsilon$, add $\epsilon$ to FIRST($A$).

### FOLLOW

FOLLOW($A$) = set of terminals that can immediately follow $A$ in some sentential form (used when the parser needs to decide between using an $\epsilon$-production and not). **Always add $ to FOLLOW(start symbol).**

**Iterative rule:** for each production $B \to \alpha A \beta$, add FIRST($\beta$)$\setminus\{\epsilon\}$ to FOLLOW($A$). If $\epsilon \in$ FIRST($\beta$) (or $\beta$ is empty), add FOLLOW($B$) to FOLLOW($A$). Iterate to fixed point.

### Building the LL(1) parsing table

Table $M[A, a]$ holds the production to use when the top-of-stack is non-terminal $A$ and the lookahead is terminal $a$.

For each production $A \to \alpha$:

- For every $a \in \text{FIRST}(\alpha) \setminus \{\epsilon\}$: set $M[A, a] = A \to \alpha$.
- If $\epsilon \in \text{FIRST}(\alpha)$: for every $b \in \text{FOLLOW}(A)$, set $M[A, b] = A \to \alpha$.

**The grammar is LL(1) iff no cell contains more than one production.** If you get a conflict, the grammar is ambiguous, left-recursive, or needs left-factoring.

### The LL(1) parsing algorithm

```
push the start symbol then $ onto stack
a := next_token()
loop:
  X := top_of_stack
  if X = $ and a = $: accept
  if X is a terminal:
    if X = a: pop; a := next_token()
    else: error
  else:  (* X is non-terminal *)
    if M[X, a] = (X -> alpha): pop; push alpha in reverse
    else: error
```

### Worked example: the expression grammar

The textbook grammar $G_3$ for arithmetic, already free of left recursion and ambiguity:

$$E \to TE', \quad E' \to +TE' \mid \epsilon, \quad T \to FT', \quad T' \to *FT' \mid \epsilon, \quad F \to (E) \mid \texttt{id}$$

**FIRST sets:** $\text{FIRST}(E) = \text{FIRST}(T) = \text{FIRST}(F) = \{(, \texttt{id}\}$, $\text{FIRST}(E') = \{+, \epsilon\}$, $\text{FIRST}(T') = \{*, \epsilon\}$.
**FOLLOW sets:** $\text{FOLLOW}(E) = \text{FOLLOW}(E') = \{), \$\}$, $\text{FOLLOW}(T) = \text{FOLLOW}(T') = \{+, ), \$\}$, $\text{FOLLOW}(F) = \{+, *, ), \$\}$.

## 1.3 Grammar transformations

**Exam-critical**

Tested in 2020-3, 2020-4, 2024-1. Two transformations to drill: **eliminating left recursion** and **left factoring**. Both turn a grammar that isn't LL(1) into one that is (when possible).

### Eliminating left recursion

**Direct left recursion** ($A \to A\alpha \mid \beta$) breaks every top-down parser: it would loop forever trying to expand $A$. The standard fix:

$$A \to A\alpha_1 \mid A\alpha_2 \mid \cdots \mid \beta_1 \mid \beta_2 \mid \cdots$$

becomes

$$A \to \beta_1 A' \mid \beta_2 A' \mid \cdots, \quad A' \to \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \epsilon$$

**Worked example.**

$$E \to E + T \mid T \quad \Longrightarrow \quad E \to TE', \quad E' \to +TE' \mid \epsilon$$

**Indirect left recursion** (e.g. $A \to B\alpha$, $B \to A\gamma$) needs to be unfolded into direct recursion first by substituting $B$'s productions into $A$'s.

### Left factoring

When two productions for the same non-terminal share a common prefix, an LL(1) parser can't decide which to take. Factor out the prefix:

$$A \to \alpha\beta_1 \mid \alpha\beta_2 \quad \Longrightarrow \quad A \to \alpha A', \quad A' \to \beta_1 \mid \beta_2$$

**Worked example.** The classic *dangling else*:

$$S \to \texttt{if}\ E\ \texttt{then}\ S \mid \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S$$

After left factoring:

$$S \to \texttt{if}\ E\ \texttt{then}\ S\ S', \quad S' \to\ \texttt{else}\ S \mid \epsilon$$

Now LL(1), *but* the dangling-else ambiguity (which `if` an `else` attaches to) remains as a parsing-table conflict; most languages resolve by preferring the shift (attach to the nearest `if`).

### The big picture

Grammar transformations don't change the *language* the grammar describes; they change the *parsing strategy* it admits. The standard pipeline:

1. Disambiguate (encode precedence and associativity into separate non-terminals — one per precedence level — with left-recursion for left-associative operators).
2. Eliminate left recursion (required for top-down parsing; LR can handle left recursion directly).
3. Left-factor (so each lookahead picks at most one production).

## 1.4 LR and SLR(1) parsing

**Exam-critical**

The harder parser flavour. You must be able to: state what an LR(0) item is, construct the DFA of item sets (via closure and goto), explain shift/reduce and reduce/reduce conflicts, run a shift-reduce parser given a parse table, and contrast LR(0) / SLR(1) / LR(1) / LALR(1).

### Why LR is more powerful

**LR parsers do a rightmost derivation in reverse** (bottom-up). They handle left recursion natively. The parser maintains a stack of grammar symbols and a state; at each step it either:

- **Shifts** the next input token onto the stack, or
- **Reduces** a stack suffix matching some production's RHS, replacing it with the LHS.

LR($k$) = Left-to-right scan, Rightmost derivation in reverse, $k$ tokens of lookahead. Almost all real languages are LR(1).

### LR(0) items

An **LR(0) item** is a production with a dot marking how much of the RHS has been parsed:

$$A \to \alpha \bullet \beta$$

Reading: "we've parsed $\alpha$; we expect $\beta$ next." An item with the dot at the end ($A \to \alpha\bullet$) means a reduction is possible.

### Closure of an item set

If $A \to \alpha \bullet B\beta$ is in the set and $B$ is a non-terminal, then for every production $B \to \gamma$, add $B \to \bullet\gamma$ to the set. Iterate to fixed point. Intuition: "if we're about to parse $B$, we're also about to parse anything $B$ starts with."

### GOTO

GOTO$(I, X)$ = closure of $\{$all items $A \to \alpha X \bullet \beta$ such that $A \to \alpha \bullet X\beta \in I\}$. This is the next item set after shifting symbol $X$.

### Building the canonical LR(0) DFA

Start state: closure$(\{S' \to \bullet S\})$ where $S'$ is a fresh start symbol (with one production $S' \to S\$$). Apply GOTO for every symbol from every state; new sets become new states. Stop at fixed point.

### From DFA to parse table: SLR(1)

- **Shift action.** If $A \to \alpha \bullet a\beta \in I_i$ and GOTO$(I_i, a) = I_j$: set ACTION$[i, a]$ = shift $j$.
- **Reduce action (SLR).** If $A \to \alpha\bullet \in I_i$: for every $a \in$ FOLLOW$(A)$, set ACTION$[i, a]$ = reduce $A \to \alpha$.
- **Accept.** If $S' \to S\bullet \in I_i$: ACTION$[i, \$]$ = accept.
- **GOTO table.** For non-terminals: GOTO$[i, A] = j$ when GOTO$(I_i, A) = I_j$.

### Conflicts

- **Shift/reduce conflict:** a state contains both a shift item ($A \to \alpha \bullet a\beta$) and a complete item ($B \to \gamma\bullet$) with $a \in$ FOLLOW$(B)$. Grammar is not SLR(1). Often arises from the dangling-else; the conventional fix is "prefer shift."
- **Reduce/reduce conflict:** two complete items in the same state with overlapping FOLLOW. The grammar is genuinely ambiguous on this input prefix.

### SLR(1) vs LR(1) vs LALR(1)

| Variant | Lookahead source |
|---|---|
| LR(0) | no lookahead; reduce on any input. Very weak. |
| SLR(1) | reduce $A \to \alpha$ only when next token $\in$ FOLLOW$(A)$. |
| LR(1) | each item carries its own lookahead set, computed per-state. Strictly more powerful than SLR(1). Often has many states. |
| LALR(1) | LR(1) with states having identical cores merged. Same power on most grammars as LR(1) for far fewer states. What `yacc/bison` use. |

**Memorable hierarchy:** LR(0) $\subsetneq$ SLR(1) $\subsetneq$ LALR(1) $\subsetneq$ LR(1) (in terms of grammars they can handle).

### The shift-reduce algorithm

```
push state 0 onto stack
a := next_token()
loop:
```

```
s := top state of stack
case ACTION[s, a]:
  shift t      -> push a, then push t; a := next_token()
  reduce A->b  -> pop 2|b| items (symbol+state pairs);
                  let s' = top state; push A;
                  push GOTO[s', A]
  accept       -> done
  error        -> syntax error
```

---

# 2 The Big Three of the Back End (25% of marks)

## 2.1 Intermediate representations and stack-based VMs

**Worth knowing**

The lecturer derives the Jargon VM from a simple interpreter through CPS-conversion, defunctionalisation, and stack-splitting. Exam questions tend to ask: "given this AST node, generate stack-machine bytecode"; "explain how function calls / closures / conditionals translate."

### Why a stack-based IR?

The case for an **intermediate representation** between AST and machine code:

- **Retargetability:** compile front-end once, write a back-end per machine.
- **Optimisation:** most optimisations are easier on a uniform IR than on either the AST or assembly.
- **Portability:** a VM-targeted IR (JVM bytecode, OCaml bytecode) can run on any platform with a host interpreter.

### The basic stack machine

Expressions become postfix sequences that push and pop on an operand stack:

| AST | Bytecode |
|---|---|
| `Int n` | `PUSH n` |
| `Add(e1, e2)` | ⟨$e_1$⟩; ⟨$e_2$⟩; `OPER ADD` |
| `Var x` | `LOOKUP x` |
| `Let(x, e1, e2)` | ⟨$e_1$⟩; `BIND x`; ⟨$e_2$⟩; `SWAP`; `POP` |
| `If(e1, e2, e3)` | ⟨$e_1$⟩; `TEST L1`; ⟨$e_2$⟩; `GOTO L2`; `LABEL L1`; ⟨$e_3$⟩; `LABEL L2` |
| `Lambda(x, e)` | `MK_CLOSURE L`; with `LABEL L`: `BIND x`; ⟨$e$⟩; `SWAP`; `POP`; `RETURN` |
| `App(e1, e2)` | ⟨$e_2$⟩; ⟨$e_1$⟩; `APPLY` |

### Compiling control flow to linear code

**Nested code** (where TEST holds two sub-instruction-lists) becomes **linear code** (where TEST holds a label and we emit the branches sequentially with GOTOs around). Two new instructions: `LABEL L` (a marker, no-op at runtime) and `GOTO L` (unconditional jump). After compilation, a second pass replaces symbolic labels with numeric code offsets.

#### Closures

A first-class function has free variables captured from its definition environment. A **closure** = code pointer + environment.

- **Compilation:** `MK_CLOSURE(label, n)` allocates a closure on the heap containing the code label and $n$ values for the captured free variables.
- **Application:** `APPLY` pops the argument and the closure, pushes the frame pointer for return, sets up a new frame, jumps to the code label.
- **Return:** `RETURN` restores the caller's frame pointer and jumps to the saved return address.

The Jargon VM puts all complex values (closures, pairs, sum-type cells) **on the heap**, keeping the stack to fixed-size words (integers, booleans, heap pointers, code pointers, stack pointers).

#### Variable access via static offsets

A variable's name is a compile-time concept. At runtime, the compiler converts each variable reference to either:

- a **stack offset** from the current frame pointer (for parameters and locals), or
- a **heap offset** into the current closure (for captured free variables).

### 2.2 Garbage collection

**Worth knowing**

Tested in 2023-2 and 2024-2. Be able to: (a) explain reference counting and its cyclic-garbage weakness, (b) describe mark-and-sweep, (c) describe copying / semi-space collection, (d) sketch generational GC and the weak generational hypothesis.

#### The root set and reachability

**An object is garbage iff it is unreachable from the root set** (registers, global variables, current stack frames). All tracing GCs share this definition; they differ in *how* they find unreachable objects.

#### Reference counting

Every object holds an integer count of incoming references. On pointer copy, increment; on overwrite or scope exit, decrement; when the count hits zero, free the object (and recursively decrement counts in fields).

**Pros:** immediate reclamation, no global pause, simple to retrofit.
**Cons: cannot collect cycles** (a self-referential structure keeps its count $\geq 1$ even when unreachable); per-pointer-update cost; cache pollution; not thread-safe without expensive atomic increments.

#### Mark and sweep

Two-phase tracing collector:

1. **Mark:** starting from roots, walk the live object graph; set a mark bit on each visited object.
2. **Sweep:** scan the entire heap; objects without the mark bit are freed (returned to the free list). Clear all mark bits.

**Pros: handles cycles**, low per-mutation cost.
**Cons:** stops the world during collection (full heap scan); causes **heap fragmentation**; mark phase touches the whole live set, hurting locality.

#### Copying / semi-space collector

Divide the heap into **from-space** and **to-space**. Allocation just bumps a pointer in from-space (extremely cheap). On GC:

1. Trace from roots; copy each live object from from-space into to-space.
2. Leave a **forwarding pointer** in the from-space copy so that any other reference to the same object during the same trace finds the new location.
3. When done, swap the roles of the spaces; from-space becomes the new free area.

**Pros:** bump-pointer allocation, automatic compaction (no fragmentation), trace cost proportional to *live* set not heap size.
**Cons: requires twice the heap**; copies all live data on every cycle; updates every pointer to copied objects.

#### Generational GC

**Weak generational hypothesis:** most objects die young. Empirically, >98% of garbage in many workloads is from the most recently allocated objects.

**Idea.** Partition the heap by object age. Run frequent, fast collections of the **young generation** (most objects there are already dead), and infrequent, slow collections of the **old generation**. Promote surviving young objects into the old generation.

**Complication.** A young object may be referenced from the old generation, so collecting the young alone requires scanning the entire old generation. The cure: a **remembered set** (or **write barrier**) that records old-to-young pointers as they are created.

### 2.3 Tail-call optimisation

**Worth knowing**

Tested in 2022-2 and 2023-2. Be able to: define a tail call, explain how TCO replaces a call+return with a jump, and explain why this is necessary for functional-language compilers to be practical.

**A tail call** is a function call that is the last action of the calling function — no further computation happens with its return value before the caller itself returns. In source code: `return f(x)` is a tail call; `return f(x) + 1` is not.

**TCO.** Instead of pushing a new stack frame for a tail call, reuse the current frame: overwrite the parameters with the new argument values and jump to the function's entry point. The call disappears at the machine level; there's just a goto.

**Effect.** A function that's *tail recursive* (recurses only in tail position) executes in constant stack space, exactly like a loop. This is the only way functional languages can implement looping — they have no `while` or `for`, only function calls.

**Example.**

```
(* Not tail recursive: each call awaits the multiplication *)
let rec fact n = if n = 0 then 1 else n * fact (n - 1)
```

```
(* Tail recursive via accumulator: the call IS the last action *)
let rec fact_acc n acc = if n = 0 then acc else fact_acc (n - 1) (n * acc)
let fact n = fact_acc n 1
```

**What the optimiser does.** Recognise the syntactic form "`return f(args)`" at the end of a function. Generate code that (a) evaluates `args` into temporaries, (b) overwrites the current frame's parameter slots with those temporaries, (c) jumps to `f`'s entry. No `call` instruction, no `ret` from `f` back through the caller.

**Mutual tail recursion** works the same way: `f` can tail-call `g` which can tail-call `f`, with both using a fixed amount of stack.

**Interaction with exception handlers.** A call placed inside a try-block is *not* a tail call — on return the handler frame must be cleaned up. (This is why some compilers don't TCO inside handlers.)

---

# 3 The Second Tier

## 3.1 Exceptions

**Worth knowing**

Tested in 2019-4, 2021-4, 2025-1. Be able to describe the VM-level mechanism: an exception pointer, handler frames, and the cleanup raise does.

**The semantics.** A `try` $e_1$ `with` $p \to e_2$ expression evaluates $e_1$. If $e_1$ yields a value $v$, return $v$. If $e_1$ raises an exceptional value $w$ matching pattern $p$, evaluate $e_2$ with $p$ bound to $w$. Otherwise, the exception continues propagating out of the `try`.

**The implementation.** Add a new VM register: the **exception pointer** (`ep`), pointing to the most recent **handler frame** on the stack. A handler frame stores:

- The address of the handler code.
- The saved frame pointer (`fp`) and code pointer (`cp`) to restore on entry to the handler.
- The saved `ep` of the enclosing handler (so we can pop our handler when leaving the `try`).

**Entering a try block:** push a handler frame; set `ep` to the new frame.

**Leaving the try normally:** restore `ep` to the saved enclosing value; the handler frame is discarded.

**Raising:** read `ep`; set `fp` and `cp` to the values saved in the handler frame; pop everything above the handler frame off the stack (this is what "unwinding" means); pass the raised value to the handler code.

**Cost.** Entering and leaving a `try` is cheap (push/pop a few words). Raising is moderately expensive (unwinds an arbitrary amount of stack). **Don't use exceptions for normal control flow in performance-sensitive code** — if exception-free execution is the common case, the design pays nothing.

## 3.2 Linking and ABIs

**Worth knowing**

Conceptual background that has appeared in passing. Expect questions about "what does the linker do?" and "compare static vs dynamic linking."

**An object file** contains: machine code (`.text`), initialised global data (`.data`), uninitialised globals (`.bss`), a **symbol table** (exported and imported names), and a **relocation table** (places in the code/data where addresses depend on the final layout). ELF is the standard format on Linux.

**The linker:**

1. Concatenates `.text` segments from all object files; ditto `.data` and `.bss`.
2. Resolves symbols: for every imported name in one object file, find a matching export in another (or in a library).
3. Performs relocations: where the code references a symbol, patch in the symbol's final address.

**Static vs dynamic linking.**

| Static | Dynamic |
|---|---|
| Library code copied into the executable at link time. | Library code loaded at run time by the OS. |
| Bigger executables. | Smaller executables. |
| Independent of installed library versions — predictable. | Library bug fixes propagate automatically — "DLL hell" is the failure mode. |
| Faster program startup. | Slower startup (loader resolves symbols on launch or lazily). |

**An ABI (Application Binary Interface)** is the contract between independently-compiled units: register usage and calling convention (which args go in which registers; who saves what), stack frame layout, data alignment, system call mechanism, name mangling (especially in C++), object file format.

### 3.3 Type systems and type checking

**Worth knowing**

Tested in 2018-4, 2020-4. Be able to: explain the role of type checking (in the front end, after parsing), describe Hindley–Milner / unification-based inference at a high level, distinguish static and dynamic typing.

**Where typing fits.** Lexer → Parser → **Type checker** → IR. The output of typing is an *annotated AST*; the type checker either accepts (every expression has a type that satisfies the language rules) or reports a type error. After type checking, the rest of the compiler can assume well-typed input.

**What a static type system buys you.**

- Many program errors caught at compile time (no `1 + "hello"`).
- Efficient code generation: no need to box every value or runtime-check every operation.
- Documentation: types describe interfaces.

**Hindley–Milner inference (very brief).** Used by ML, OCaml, Haskell. Walk the AST; for each expression, generate a type with fresh variables for unknown parts and a set of equality constraints. Solve constraints by **unification**. Generalise type variables that don't appear in the surrounding context to produce *polymorphic* types.

**Dynamic typing** (Python, JavaScript): every value carries a type tag; every operation checks tags at runtime; "type errors" become runtime exceptions. Easier to write, harder to debug, slower without sophisticated JIT compilation.

## 3.4 Compiler optimisations

**Worth knowing**

The big lecture topic. The exam usually asks "define optimisation X and give an example," not "implement it."

### What makes an optimisation valid

An optimisation transforms a program while preserving *observable behaviour*: same effects, same termination behaviour, same return value. The harder language has, the more subtle preservation becomes (e.g. floating-point associativity is *not* a valid optimisation in IEEE 754).

### The classic catalogue

- **Constant folding:** evaluate constant expressions at compile time. `2 + 3` → `5`.
- **Constant propagation:** replace a variable with its known constant value.
- **Algebraic simplification:** `x * 1` → `x`, `x + 0` → `x`, `x - x` → `0`. Care: `x * 0` → `0` only if `x` has no side effects.
- **Common subexpression elimination (CSE):** `a*b + a*b` → `let t = a*b in t + t`.
- **Dead-code elimination:** remove computations whose result is never used.
- **Inlining:** replace a function call with the function body. Enables many subsequent optimisations; can bloat code. The compiler heuristic decides which to inline based on size, call count, and exposed opportunities.
- **Tail-call optimisation:** as above.
- **Loop-invariant code motion:** hoist a computation that doesn't depend on the loop variable out of the loop.
- **Strength reduction:** replace a costly operation with a cheaper one. `x * 2` → `x << 1`; multiply-by-loop-induction-variable → add.
- **Peephole optimisation:** pattern-match small windows of generated assembly and replace with shorter equivalents (e.g. `push X; pop X` → nothing; `mov rax, rax` → nothing).

### Optimisation under undefined behaviour

**Two principles for languages like C**:

1. The compiler imposes no constraints on the behaviour of programs that have undefined behaviour (UB).
2. The compiler may therefore *assume* the input program does not exhibit UB.

Consequence: optimisations can transform a UB-exhibiting program in surprising ways. For example, signed integer overflow is UB in C, so the compiler may assume `i + 1 > i` holds for all `int i` — and delete a loop bound check that would only fire if it didn't.

## 3.5 Data types and records

**Worth knowing**

Tested in 2019-3, 2021-3. Be able to: describe how tuples / records / sum types lay out in memory, and how method dispatch works in OO languages.

**Tuples and records.** A flat block on the heap, with each field at a known compile-time offset. The compiler computes offsets from declaration order plus alignment requirements; access is a single load.

**Sum types (variants).** A **tag** word identifying which case the value is, followed by the case's payload. Pattern matching becomes a switch on the tag.

**Arrays.** Contiguous memory, optionally preceded by a length word. Bounds-checking on each access in safe languages (cheap and removable by the optimiser when bounds are statically known); absent in unsafe languages like C.

**Method dispatch in OO languages.**

- **Static dispatch** (Java's `final`, C++'s non-`virtual`): the call target is decided at compile time. Cheap; no indirection.
- **Dynamic dispatch** (Java's normal methods, C++'s `virtual`): each object's header points to a **vtable** (per-class array of function pointers); a method call indirects through the vtable. Costs one extra load and an indirect call; the indirect call also defeats inlining.

**Multiple inheritance** (C++): more complex — vtables per ancestor class, with offsets to adjust `this` when casting between super- and subobjects.

## 3.6 Bootstrapping

**Worth knowing**

Background topic. Not specifically tested in recent years but underpins "how was the OCaml compiler compiled?" — a good interview-style question.

**The puzzle:** the OCaml compiler is itself written in OCaml. How was the first OCaml compiler compiled?

**Bootstrap chain (general):**

1. Write a small subset compiler ($lang_0$) in some other language (e.g. C).
2. Use it to compile a larger subset compiler ($lang_1$) written in $lang_0$.
3. Use that to compile the full compiler written in $lang_n$.
4. Henceforth, each release is compiled by the previous release.

**Ken Thompson's "Reflections on Trusting Trust".** An attacker who modifies the compiler binary to inject a backdoor whenever it compiles a login program — *and* the same compiler binary to inject the backdoor-injecting code whenever it compiles a compiler — can hide the attack such that the source contains no trace. The lesson: a compiler binary is only as trustworthy as the chain of binaries that produced it.

---

# 4 The one-page exam checklist

**Exam-critical**

If you have one hour to revise, do these in order:

1. Be able to apply Thompson's construction to a small regex to produce an NFA.
2. Be able to do the subset construction with $\epsilon$-closures to produce a DFA from an NFA.
3. Be able to compute FIRST and FOLLOW sets for a given grammar.
4. Be able to build the LL(1) parsing table, identify any conflicts, and run the parser on a string.
5. Be able to eliminate left recursion and left-factor a small grammar.
6. Be able to construct the LR(0) item-set DFA: closure, GOTO, build the canonical collection.
7. Be able to convert that DFA to an SLR(1) parse table using FOLLOW sets, identify shift/reduce and reduce/reduce conflicts.
8. Be able to run a shift-reduce parser given a parse table.
9. Be able to translate a small expression (with conditionals and functions) into stack-machine bytecode, including labels for branches.
10. Be able to define "tail call" and explain TCO at the machine level.
11. Be able to describe reference counting, mark-and-sweep, copying, and generational GC — with one pro and one con of each.
12. Be able to describe the implementation of exceptions (handler frames + exception pointer + stack unwinding).
13. Be able to name 5 standard optimisations and give an example of each.

*Good luck.*